Portland State
Computer Science

Heavily based on material from:
Murach's MySQL (3rd Edition), by Joel Murach

Normalization

The process of organizing the columns (attributes) and tables (relations) of a relational database to **minimize data redundancy**.

Normalization involves **decomposing** a table into less redundant (and smaller) tables without losing information; defining foreign keys in the old table referencing the primary keys of the new ones.

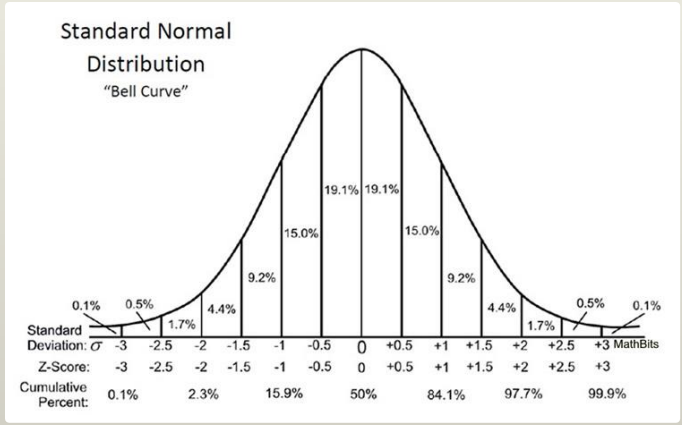
The objective is to isolate data so that additions, deletions, and modifications of an attribute can be made in just one table and then propagated through the rest of the database using the defined foreign keys.

Normalization

There are 10 normal forms, but ***typically*** only 3 are applied.

Though normalization reduces redundancy, it can cause storage and maintenance problems.

I found references for **10** normal forms (not including "unnormalized form").



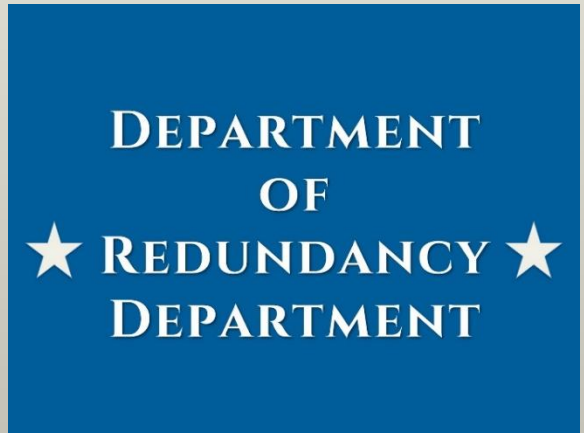
https://en.wikipedia.org/wiki/Database_normalization

The first three normal forms

Normal form	Description
First (1NF)	The value stored at the intersection of each row and column must be a scalar value, and a table must not contain any repeating columns.
Second (2NF)	Every non-key column must depend on the entire primary key.
Third (3NF)	Every non-key column must depend only on the primary key.

Note

- Most database designers stop at the third normal form.



The next four normal forms

Normal form	Description
Boyce-Codd (BCNF)	A non-key column can't be dependent on another non-key column.
Fourth (4NF)	A table must not have more than one *multivalued dependency*, where the primary key has a one-to-many relationship to non-key columns.
Fifth (5NF)	The data structure is split into smaller and smaller tables until all redundancy has been eliminated.
Domain-key (DKNF) or and Sixth(6NF) is the set	**Every constraint on the relationship is dependent only on key constraints** ***domain*** **constraints, where a domain** of allowable values for a column.

Two tables that need to be normalized

A table that contains repeating columns

vendor_name	invoice_number	item_description_1	item_description_2	item_description_3
Cahners Publishing	112897	VB ad	SQL ad	Library directory
Zylka Design	97/552	Catalogs	SQL flyer	NULL
Zylka Design	97/553B	Card revision	NULL	NULL

A table that contains a column for each invoice item. Ick!

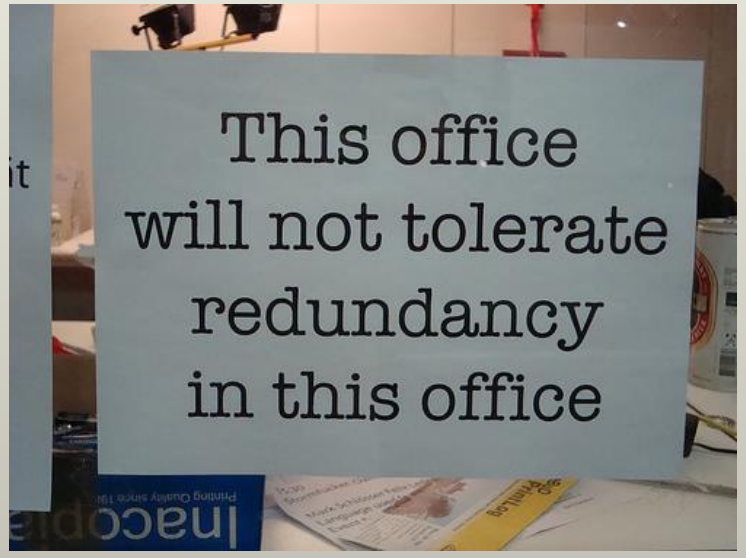
A table that contains redundant data

vendor_name	invoice_number	item_description
Cahners Publishing	112897	VB ad
Cahners Publishing	112897	SQL ad
Cahners Publishing	112897	Library directory
Zylka Design	97/522	Catalogs
Zylka Design	97/522	SQL flyer
Zylka Design	97/533B	Card revision

A table that contains a description for each invoice item. More ick!

Terms

- Normalization
- Data redundancy
- Unnormalized data structure
- Normalized data structure
- Normal forms



Benefits of Normalization

1. The process of organizing the columns (attributes) and tables (relations) of a relational database to **minimize data redundancy**.
2. More tables, and **each table can have an PK index**, the database has more PK indexes. That makes **data retrieval more efficient**.
3. Each table contains information about a single entity, and **each index has fewer columns** (usually one) and fewer rows. That makes data retrieval and **insert, update, and delete operations more efficient**.
4. Each **table has fewer indexes**, which makes **insert, update, and delete operations more efficient.**
5. **Simplified maintenance and reduced storage**, due to reduced data redundancy, i.e. efficiency.

Unnormalized invoice data

The invoice data with a column that contains repeating values

vendor_name	invoice_number	item_description
Cahners Publishing	112897	VB ad, SQL ad, Library directory
Zylka Design	97/522	Catalogs, SQL Flyer
Zylka Design	97/533B	Card revision

Notice the comma separated values in the column.

Application developers LOVE to do this kind of thing

The invoice data with repeating columns

vendor_name	invoice_number	item_description_1	item_description_2	item_description_3
Cahners Publishing	112897	VB ad	SQL ad	Library directory
Zylka Design	97/552	Catalogs	SQL flyer	NULL
Zylka Design	97/553B	Card revision	NULL	NULL

What happens when we have an order with 4 items?

The invoice data in first normal form

	vendor_name	invoice_number	item_description
▶	Cahners Publishing	112897	VB ad
	Cahners Publishing	112897	SQL ad
	Cahners Publishing	112897	Library directory
	Zylka Design	97/522	Catalogs
	Zylka Design	97/522	SQL flyer
	Zylka Design	97/533B	Card revision

After, **a separate row for each value**.

For a table to be in first normal form, its **columns must not contain repeating values**. Each column must contain a single scalar value.

First normal form often has repeating values in rows.

Before, with comma separated values.

	vendor_name	invoice_number	item_description
▶	Cahners Publishing	112897	VB ad, SQL ad, Library directory
	Zylka Design	97/522	Catalogs, SQL Flyer
	Zylka Design	97/533B	Card revision

The invoice data in first normal form with keys added

Before

invoice_id	vendor_name	invoice_number	invoice_sequence	item_description
1	Cahners Publishing	112897	1	VB ad
1	Cahners Publishing	112897	2	SQL ad
1	Cahners Publishing	112897	3	Library directory
2	Zylka Design	97/522	1	Catalogs
2	Zylka Design	97/522	2	SQL flyer
3	Zylka Design	97/533B	1	Card revision

The keys added to order the items in the invoice.

For a table to be in second normal form, **every non-key column must depend on the entire primary key**. If a column does not depend on the entire primary key, it indicates that the table contains data for more than one entity.

vendor_name	invoice_number	item_description
Cahners Publishing	112897	VB ad
Cahners Publishing	112897	SQL ad
Cahners Publishing	112897	Library directory
Zylka Design	97/522	Catalogs
Zylka Design	97/522	SQL flyer
Zylka Design	97/533B	Card revision

The duplicate data representing the vendor name has been removed.

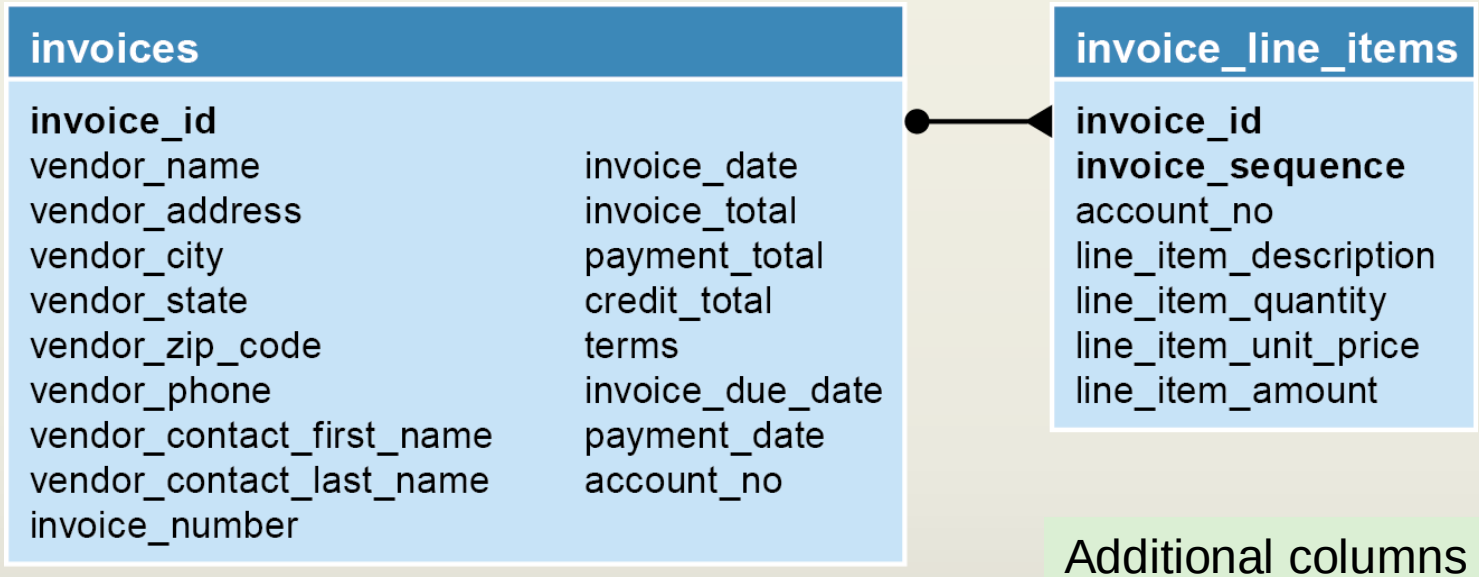
	InvoiceNumber	VendorName	InvoiceID
1	112897	Cahners Publishing	1
2	97/522	Zylka Design	2
3	97/533B	Zylka Design	3

	InvoiceID	InvoiceSequence	ItemDescription
1	1	1	VB ad
2	1	2	SQL ad
3	1	3	Library directory
4	2	1	Catalogs
5	2	2	SQL flyer
6	3	1	Card revision

A new table representing the invoice_line_items has been created.

This allows the vendor name to only occur once, in the invoices table.

The A/P system in second normal form



Additional columns are shown here.

Questions about the structure

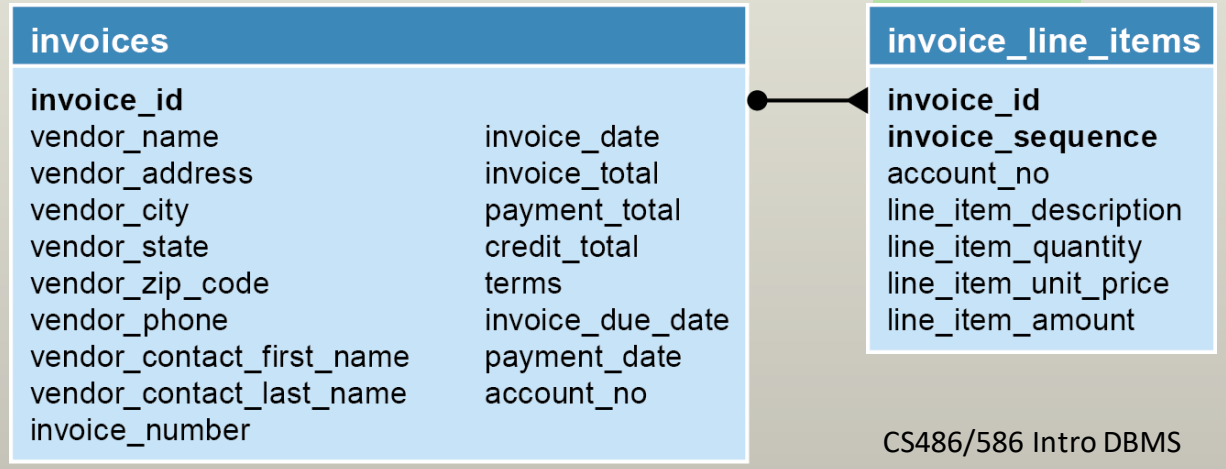
1. Does the vendor information (Vendor_Name, Vendor_Address, etc.) depend only on the Invoice_ID column?

2. Does the Terms column depend only on the Invoice_ID column?

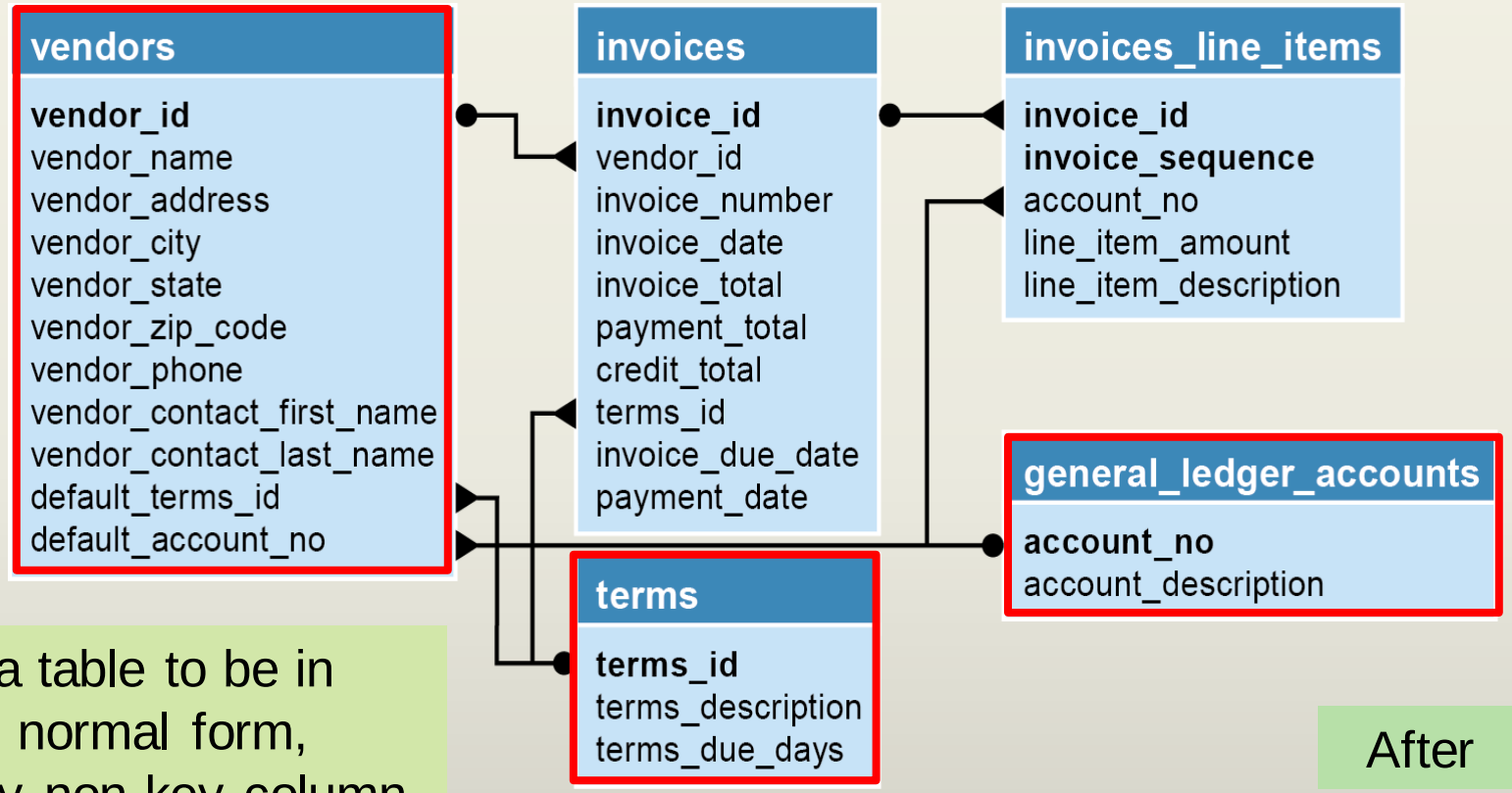
3. Does the Account_No column depend only on the Invoice_ID column?

4. Can the Invoice_Due_Date and Invoice_Line_Item_Amount columns be derived from other data?

Before



The accounts payable system in third normal form

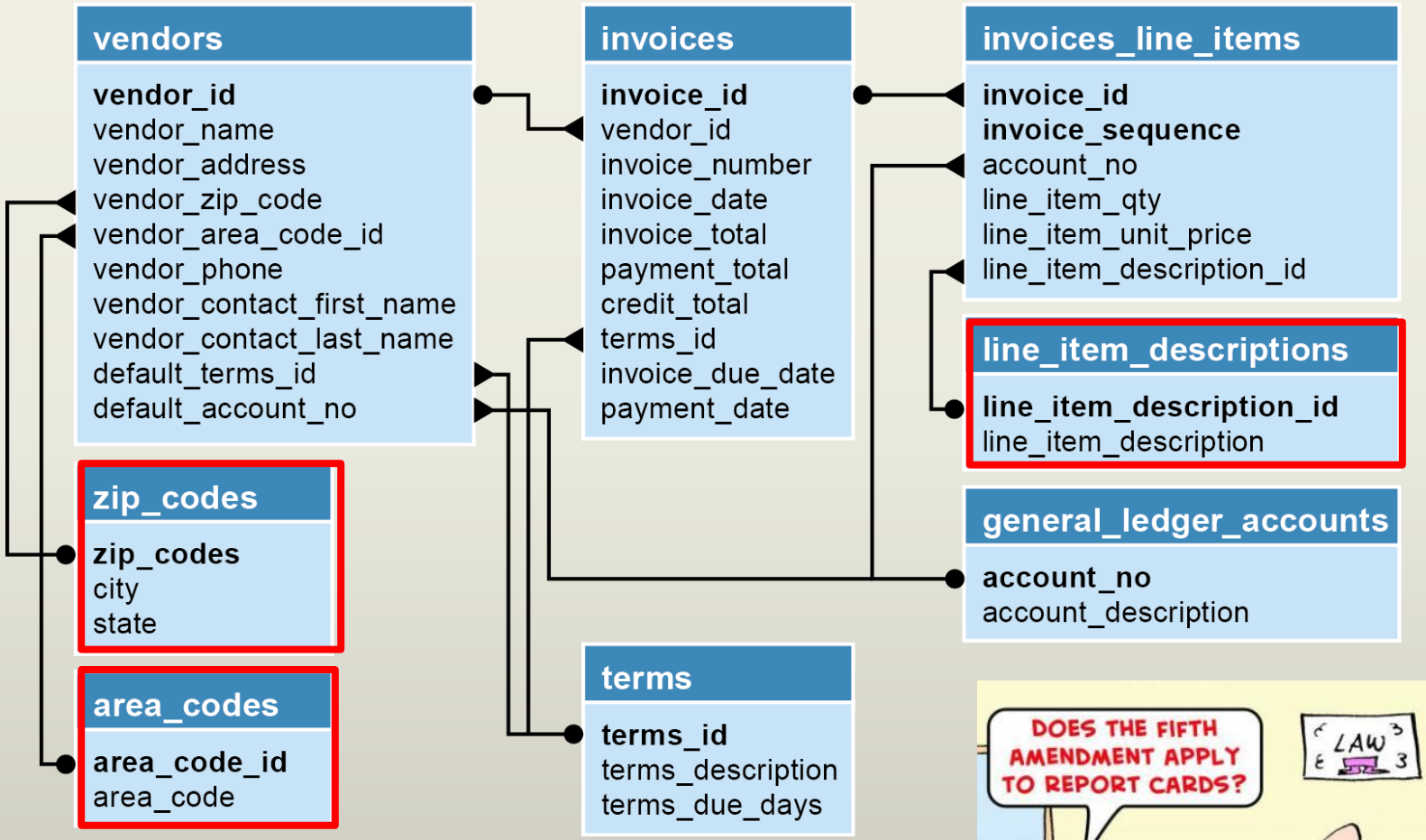


For a table to be in third normal form, every non-key column must depend only on the primary key.

After

We created the Vendors, Terms, and GLAccounts tables and moved duplicative columns into the new tables.

The accounts payable system in fifth normal form



If we wanted to go to fifth-normal form, we'd need to add 3 new tables.

Do we always normalize?

An objective of **normalization** is to reduce redundant data stored in the database.

In a **transactional database**, normalization helps us with data consistency and correctness.

What are the conditions that may cause us to **denormalize** a database?

When to denormalize

1. When a column from a joined table is used **repeatedly** in search criteria.
2. If a table is **updated infrequently**.
3. Include columns with derived values when those values are used frequently in search conditions.

Denormalization is a strategy used on a previously-normalized database to **increase performance**.

The idea behind it is to add redundant data where we think it will help us the most.

Other reasons to denormalize?

Highly normalized databases can often require many-Many-**MANY** joins before you can get out any useful data. Denormalization can reduce the length of the SQL statements.

Denormalization is often done on reporting databases.